Further Lean proof simplifications

Coefficient composition, scalar thresholds, and an optional computable quotient

Technical review for Vladimir Reshetnikov

26 October 2026

Abstract

The retained changes compose the existing linear coefficient estimate to prove a square bound, and replace two scalar exponential inductions by power normalization. The complete source snapshots for these changes decrease from 752 to 734 physical lines, including the relocated lemma and compatibility wrapper. This 18-line reduction is deliberately modest: the main improvement is the elimination of a repeated double-sum estimate and two local inductions.

Both coefficient modules were checked completely, before and after the change. The scalar proofs were checked as exact original and replacement declarations; their complete containing modules were not rebuilt. An optional change uses the existing arithmetic decoder to compute the quotient tile index. It was checked as a complete module with its original checks, but adds 13 source lines and is presented as a representation improvement.

The proof-source pin is `29936089835c0c55cca81edc86b4fb0c70ce34d5`. The final exclusion cutoff is `09d65a93c9df8684c1a226341d6a5120623d1cdb`: 23 placed packets a–w and four incoming archives. A polynomial-identity proposal and its finite-product cleanup were removed when packet w was found to contain the same ideas. The archive omits those candidate proofs and their savings.

1 Results, scope, and recommended order

The most attractive first change is the coefficient estimate. It has a short mathematical explanation, preserves the exact statement of every old theorem, and removes a second proof of a bound that already follows from the linear case. The two numerical thresholds are independent local cleanups. The computable quotient is optional and should be considered separately from the source reductions. The primary recommendations comprise two mechanisms with three concrete target proofs; they are not three separate architectural frameworks.

These measurements do not count the many earlier proposed reductions. They are not a benchmark of a hypothetical checkout containing all earlier packets. The earlier ideas were treated as the assumed conceptual baseline when deciding which mechanisms to retain; the supplied patches and measurements refer to the precise, reproducible source snapshot. This distinction matters because many earlier packets were proposals rather than one mutually compatible integrated tree.

All primary changes preserve the executable definitions they touch indirectly, the literal coefficients, the machine, and the original public theorem statements. The coefficient patch adds an earlier theorem and keeps the old linear-bound name as an alias. The scalar patches change only proof bodies. The optional quotient patch changes the definition that produces an index; a separate comparison file proves equality to the old definition.

Change	Before	After	Saved	Checking scope
Coefficient bound, two modules	328	316	12	Both complete original and candidate modules; six public axiom comparisons
Two exponential thresholds, containing source snapshots	424	418	6	Exact old/new declarations; complete native modules not rebuilt
Primary changes combined	752	734	18	No full repository build claimed
Optional quotient decoder	147	160	−13	Complete candidate, original checks, equality to the legacy word

Table 1: Physical source lines at the proof-source pin. Added helper statements, documentation, and compatibility declarations are included.

1.1 What the freshness check changes

The repository advanced during review. The initial three incoming ZIPs were placed as packets s, t, and u; the next closing check added packet v. A final check found packet w and four further incoming archives. The review is frozen at the final exclusion cutoff above, so later repository changes are outside this stated scope.

Packet v explicitly covers the finite-last equivalence for the machine state number. Packet w explicitly proposes the polynomial-before-evaluation proof for `RefinedPair`, including injective exponent expansion and an accompanying `finTwoArrowEquiv Bool` cleanup. Both were independently developed during this review but are now excluded. Better compiler receipts or a smaller variation do not make an already proposed mechanism new.[1]

Among the four retained primary target source files, the later revisions change `NativeHalfBinomial.lean`. The entire `seventy_two_mul_lt_two_pow` declaration remains byte-identical to the proof-source version. Apply the small patch to a newer file; the older whole-file snapshot is supplied for inspection, not as an overwrite of earlier work. The optional decoder patch also composes textually with the new incoming helix-proof simplification, preserving its 13-line cost; compilation of that combined hypothetical tree is not claimed.

2 Non-repetition: what was treated as already done

The audit includes the substantive proposals in the placed packets, their supplements, notes, candidate decisions, and the relevant incoming index. Optional, deferred, unimplemented, and screened suggestions were treated as prior ideas. Adoption status was never used as evidence of novelty.

The first completed stage covers 22 packets a–v and 55 principal TeX files, totaling 896,675 bytes of principal manuscript text, with 129 overlapping exclusion records. Those are historical stage totals, not the size of the final corpus. The closing supplement adds packet w and four incoming archives, making 27 collections at the final cutoff. Their principal reports, technical notes, optional routes, and future questions were reviewed against each retained mechanism. Exact archive and manuscript hashes and candidate-specific decisions are supplied in `audit/closing/`.

The audit does not claim a semantic review of every unchanged source snapshot bundled with those collections, nor every historical manuscript transitively linked from the incoming index. Adoption status is not a novelty test: optional, deferred and unimplemented suggestions count as prior ideas. The retained changes have comparisons with their nearest predecessors, rather than merely negative filename searches.

For example, the following were excluded:

- Generic recursor congruence; scanner iteration, decoder/fuel fusion, validated histories, and primitive-recursive composition automation.
- Assignment naturality, witness-layout equivalences, shared residual sums, DAG invariants, generated circuit-step interfaces, and common circuit tails.
- Finite monomial majorant certificates, exact local pair fibres, support envelopes, canonical carries, and point-mass convolution for rotations.
- Shared Pell ratio estimates, fundamental-unit and parity reuse, signed congruence interfaces, integer-root descent, and stronger native rank bounds.
- Internal formula specification normalization, definable-induction transport, certificate closure, and removal of unnecessary internal inductions.
- Period-lattice interfaces, interval propagation from a seed, least exits, shared-edge Wang rows, and the later state-number/action/row proposals.
- Polynomial-core equality before evaluation, the finite-sign equivalence cleanup, fixed-index Pell anchoring, selector color moments, and canonical operating-row recognition.

Retained change	Closest earlier work	Remaining concrete difference
Compose coefficient estimates	Packet c's symbolic majorants; packet t's exact pair fibres	Iterates the already proved uniform linear multiplier bound; no table certificate or isolated fibre is needed
Remove scalar inductions	Pell ratio and rank simplifications; symbolic-power advice	Changes two unchanged scalar threshold proofs by a new variable or shifted exponent
Compute allowed-tile indices	Row-membership sharing; existing `recoveredIndex` and its injectivity uses	Uses the known decoder to replace the data-producing choice and proves equality to the old quotient word

The earlier exclusions are also informative. A period-one orbit proof of `row_zero_of_helix` looked new until the supplementary period-lattice proposal in packet n was inspected. A short sentinel-decoder proof of `stateNumber_injective` was also rejected: decoder-based injectivity was already a prior mechanism, and packet v subsequently supplied the exact finite-state equivalence direction. Neither is presented as an additional new saving here.

3 Compose the coefficient bound instead of expanding a square

3.1 The repeated argument

Let

$$E(X) = \sum_i u_i X^{w_i}, \qquad S = \sum_i |u_i|,$$

where $u_i \in \mathbb{Z}$ and the positions $w_i \in \mathbb{N}$ are arbitrary. Suppose every coefficient of a polynomial p has absolute value at most H, with $H \geq 0$. The existing theorem `SelectiveCoefficientBounds.abs_coeff_square_mul_le` states

$$\left|[X^n](E(X)^2 p(X))\right| \leq HS^2.$$

Its current proof expands E^2 into a double sum, applies the triangle inequality at both levels, bounds the monomial products individually, and reconstructs the square of a sum.[4]

The later `SplitCoefficientBounds` module already proves the linear estimate

$$\left|[X^n](E(X)D(X))\right| \le HS \quad \text{whenever} \quad \forall j,\ |[X^j]D(X)| \le H.$$

That is precisely the operation needed twice. The module order currently hides it from the earlier square proof.[5]

3.2 The replacement

Move the existing linear argument into the earlier coefficient-bound module under the name `abs_coeff_encoded_mul_le`. Keep the old theorem `SplitCoefficientBounds.abs_coeff_linear_mul_le`, with its full original signature, as a one-line alias.

First apply the linear bound to Ep, obtaining the uniform bound HS. Then apply it to $E(Ep)$, obtaining $(HS)S = HS^2$. Associativity identifies $E(Ep)$ with E^2p. The actual replacement body is:

```
have hsum : 0 ≤ ∑ i, |u i| :=
  Finset.sum_nonneg (fun i _ => abs_nonneg _)
have hlinear := abs_coeff_encoded_mul_le w u p H hH hp
have h := abs_coeff_encoded_mul_le w u (encoded w u * p)
  (H * ∑ i, |u i|) (mul_nonneg hH hsum) hlinear n
simpa only [pow_two, mul_assoc] using h
```

The displayed code wraps one line for readability; the supplied source has five nonblank proof-body lines. The old proof has 25. Counting the relocated linear argument, its full theorem statement and documentation, and the compatibility alias, the two complete files decrease by 12 physical lines.

3.3 Why the hypotheses are exactly sufficient

No distinctness condition on w is introduced. If two positions collide, the coefficients combine before the absolute value is taken; the triangle inequality still supplies the same uniform bound. This is an estimate on arbitrary encoded words, not the extraction of one isolated coefficient.

The cases $H = 0$ and $S = 0$ also need no special branches. The first application produces a valid bound, and `mul_nonneg hH hsum` gives the second application's nonnegativity premise. No cancellation, division, positive input, or nonzero word is assumed. The original factor HS^2 is retained exactly.

One can understand the proof as a bound on multiplication by E: it multiplies a uniform coefficient bound by at most S. No normed-space structure is required. Introducing such a structure solely for these two theorems would add unnecessary infrastructure. The existing elementary estimate already provides the right interface.

3.4 Why this remains after the prior proposals

Packet c's majorant-vector idea concerns symbolic bounds for supplied finite coefficient tables. Packet t's pair-fibre result identifies an exact coefficient when the contributing pair set is known. Both can be present without making this square proof a composition of the existing linear estimate.

The new proof reuses the earlier mathematics rather than inventing a second coefficient normalizer. Its incremental content is the lower placement of this specific already proved operation and its two applications at the square theorem. The public Split theorem stays available, so existing users do not need to migrate.

The closing incoming archives do not change this comparison. Their color-moment proposal reconstructs numerical selector masses from small statistics, and their geometric-sum proposal replaces specific range-bound inductions. Neither composes the existing uniform bound for multiplication by an encoded polynomial.

3.5 Checking and adoption

Both original modules and both complete modified modules compiled against the pinned repository sources. A separate check exercises the helper and exact square statement. Six public endpoints were axiom-audited before and after; their dependency sets agree:

$$\{\texttt{propext},\ \texttt{Classical.choice},\ \texttt{Quot.sound}\}.$$

The patch changes no data definition, import, or old theorem type. Its integration unit is the two-file patch; applying only the earlier-file half loses the compatibility cleanup, while applying only the later-file half leaves the helper unavailable.

4 Remove two scalar exponential inductions

The earlier Pell work gives the difficult ratio, index, and rank arguments. The following two scalar threshold theorems remain useful after those results are assumed applied. Both currently use a local induction whose conclusion follows from normalizing the bases of the powers.

4.1 Base sixteen: one quadratic inequality

`NativePellPower.sixteen_pow_bound` states

$$r \geq 2 \quad \Longrightarrow \quad 2^{2r+2} + 2 \leq 16^r.$$

Put $t = 4^r$. Monotonicity gives $t \geq 16$, while

$$2^{2r+2} = 4t, \qquad 16^r = t^2.$$

The remaining inequality is $4t + 2 \leq t^2$. For example,

$$t^2 - 4t - 2 = (t - 16)(t + 12) + 190 \geq 0$$

for $t \geq 16$. The checked Lean proof lets `nlinarith` handle the last polynomial comparison:

```
have ht : 16 ≤ 4 ^ r := by
  simpa using
    (Nat.pow_le_pow_right (by decide : 0 < 4) hr)
rw [pow_add, pow_mul, show (2 : ℕ) ^ 2 = 4 from rfl]
rw [show (16 : ℕ) = 4 ^ 2 from rfl, ← pow_mul,
  Nat.mul_comm 2 r, pow_mul]
nlinarith
```

The original threshold $r \geq 2$ and theorem type are retained. This does not ask the nonlinear tactic to reason about arbitrary exponential functions: the exponentials have already been represented by one natural number. [6]

4.2 The coefficient seventy-two: shift the threshold

`NativeHalfBinomial.seventy_two_mul_lt_two_pow` states

$$r \geq 4 \quad \Longrightarrow \quad 72r < 2^{2r+1}.$$

Write $r = 4 + n$. Then

$$2^{2r+1} = 512 \cdot 4^n.$$

Mathlib's `Nat.lt_pow_self` gives $n < 4^n$, hence $n + 1 \leq 4^n$. Consequently,

$$72n + 288 < 512n + 512 \leq 512 \cdot 4^n.$$

After the power identity is normalized, only a linear arithmetic argument remains:

```
obtain ⟨n, rfl⟩ := Nat.exists_eq_add_of_le hr
have h : n < 4 ^ n :=
  Nat.lt_pow_self (by decide : 1 < 4)
rw [show 2 * (4 + n) + 1 = 9 + 2 * n by omega,
  pow_add, pow_mul]
norm_num
omega
```

Again, this does not strengthen a hypothesis or replace the displayed constants. It proves the exact original statement. In the actual source, each old proof body has eight nonblank lines and each replacement has five. There is no new helper to amortize. The two containing source snapshots decrease by six physical lines. [7]

4.3 What was checked, and what was not

The exact original and replacement declarations both compiled with Lean 4.32.0 and the repository's mathlib revision. Their theorem headers are byte-identical. The complete candidate `NativePellPower` and `NativeHalfBinomial` modules were not rebuilt; the archive labels this boundary explicitly.

The first theorem retains the three standard axioms. The second changes its theorem-local dependency set from {`propext`, `Classical.choice`, `Quot.sound`} to {`propext`, `Quot.sound`}. This is an audit of the exact isolated declaration, not a claim that the containing module or later Pell endpoints become choice-free.

The counterfactual check is also concrete. Packet u's candidate `NativeHalfBinomial.lean` leaves the seventy-two threshold theorem unchanged and continues to use it. The closing repository revision also leaves that entire declaration unchanged. Thus the new scalar proofs are not simply a reimplementation of the earlier ratio or rank suggestions.

The closing structural archive's fixed-index Pell proposal is also distinct: it bounds a sequence at index six and then uses index monotonicity. Here the two exact scalar statements are proved by normalizing their exponential bases, respectively by shifting the exponent. The incoming constructor-threshold and geometric-sum ideas have different premises and conclusions. All of those earlier mechanisms remain part of the assumed baseline.

5 Optional: compute the allowed tile's index

5.1 Use the existing inverse for data

`WangTiles942Permutation` already supplies the arithmetic decoder `recoveredIndex` and proves

$$\text{recoveredIndex}(\text{tile}(i)) = i.\text{val}.$$

The quotient construction later chooses an index from the existential statement that an allowed tile is a table tile. The current `quotientWord` is therefore declared `noncomputable`, although its index is already determined by a known decoder.[8, 9]

For an allowed tile t, first derive $\text{recoveredIndex}(t) < 942$. This uses an allowedness witness only to prove the bound: if $t = \text{tile}(i)$, the existing decoder theorem identifies the number with $i.\text{val}$. Define

```
def indexOfAllowed (t : Tile) (ht : Allowed t) : Fin 942 :=
  (recoveredIndex t, recoveredIndex_lt_of_allowed ht)
```

The numerical output of this definition is the explicit decoder. The proof component certifies its range. The quotient word can now apply this definition to $T(-z.\text{val}, 0)$, and its public type stays the same.

The new right-inverse theorem proves

$$\text{tile}(\text{indexOfAllowed}(t, h_t)) = t.$$

Using the already proved injectivity of `tile`, the new quotient word equals the legacy choice-based quotient word at every position. The argument does not need a positive modulus: it is valid even for ZMod 0. The existing marked-word conclusion is then transported through this exact equality.

5.2 An explicit cost, not a source-saving claim

The complete module grows from 147 to 160 lines. This is not included in the 18-line primary reduction. The benefit is that the data-producing expression has a concrete arithmetic normal form. In particular, the compatibility prototype checks Start, initLeft, and initHead indices by `rfl`, obtaining 0, 927, and 928.

There is also an important definitional distinction. Equality to the old function is proved extensionally; the new function is not promised to be definitionally equal to the old `Classical.choose` expression. Code that deliberately unfolds `quotientWord` should be reviewed before adopting this optional change. The supplied original checks passed against the complete candidate module.

Removing choice from the data expression does not remove `Classical.choice` from the theorem's transitive axiom set. The existing allowedness and table-correctness proofs already depend on it. The report makes no evaluation-speed or module-wide axiom-reduction claim.

5.3 Checking and relevance to the earlier packets

The complete changed quotient module and the original unmodified checks compiled. A separate file imports the baseline module and defines the new word alongside it, proves equality to that exact

legacy definition, and transfers `MarkedWord`. The original checks cover signed addresses, negative time/coordinates, noncoprime dimensions, and modulus one under their stated hypotheses. No full repository build or later canonical-word client build is claimed.

The earlier packets discuss row-membership sharing and already use structural index recovery to prove injectivity. Neither is credited again here. The incremental proposal is to use the existing inverse as the actual data-producing quotient constructor and certify compatibility with the old choice-based constructor. It is useful only if this explicit representation is worth the additional helper lines.

The closest closing proposal factors band matching and binding for the existing quotient word, explicitly preserving its choice-based definition. The canonical-row recognition proposals identify whole tiles with the operating-row construction. Neither replaces the existential index choice by the existing arithmetic decoder. The archive records the exact comparison and the textual composition check with the incoming helix proof.

6 Integration, evidence, and reproduction

6.1 Use focused patches

The archive provides two primary patches. Apply the coefficient patch as a two-file unit. The scalar patch changes only two small proofs and should be applied after earlier native-Pell edits. The optional quotient patch is separate.

The complete candidate files are included to make the results reviewable without reconstructing a diff. They are snapshots against the proof-source pin, not a request to replace newer source files wholesale. The unchanged-theorem freshness check for `NativeHalfBinomial` is included specifically because the repository advanced during the review.

No upstream branch, commit, pull request, or incoming file was written by this review. Integration is represented by the downloadable patches and artifacts.

6.2 Verification levels

1. **Complete module checks:** both original and replacement coefficient modules. Own-project dependencies were rebuilt from the pinned source; external package objects were reused at the exact pinned revision.
2. **Exact declaration checks:** both original and replacement scalar thresholds. These establish the Lean correctness of their proof bodies, while leaving complete-module integration as a remaining check.
3. **Optional complete-module and client checks:** the quotient module and its existing checks, plus an additive comparison to the original definition.
4. **Source and provenance checks:** unchanged old theorem headers, helper-inclusive line counts, hashes, patch application, and the closing exclusion/target-source comparison.

The supplied evidence distinguishes these levels rather than treating one successful prototype as a clean build of the entire project. There are no admitted theorem bodies or newly declared axioms in the delivered candidates. The reported axiom sets are those printed by Lean, including the standard axioms where used.

The compiler version is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The mathlib revision is `81a5d257c8e410db227a6665ed08f64fea08e997`, matching the repository manifest. Missing mathlib dependencies were compiled locally with mathlib's `autoImplicit=false` and `maxSynthPendingDepth=3` options. Runtime paths in the raw logs record this particular run; they are not required directory names for a later replay.

6.3 Artifact map

`report.tex` and `report.pdf`
This report and its editable LaTeX source.

`patches/`
The two primary integration patches and the separate optional decoder patch.

`overlay/`
Complete primary candidate modules at the proof-source pin.

`optional/`
The full optional quotient candidate.

`checks/`
Exact scalar before/after declarations, coefficient checks, and the additive old/new quotient comparison. The latter must be run against the baseline.

`evidence/`
Successful checking receipts, axiom comparisons, source measurements, and source/freshness hashes.

`audit/`
The initial a–v exclusion catalogue, plus the closing a–w/four-incoming source-hash verification and candidate-specific decisions.

`scripts/`
Patch inspection/replay assistance and integrity checking. The README describes the intended baseline and candidate phases.

7 Further questions that follow from these changes

7.1 How far should coefficient composition go?

The same linear estimate immediately suggests bounds for E_1E_2p, with factor $(\sum_i |u_i|)(\sum_j |v_j|)$, and for E^kp, with factor S^k. These are follow-ups to the multiplication-bound interface, not additional measured savings in this packet. A sensible adoption test is to find an actual consumer that currently repeats the multi-sum argument. If there is only one square client, the present two applications are simpler than a large new hierarchy of norm instances.

7.2 Can threshold normalization be recognized systematically?

The two checked scalar examples have a common practical pattern: several exponential terms share a base after normalization, or a lower threshold allows an exponent shift that leaves a standard growth inequality. A future tool could propose these changes, but the final certificate should expose the

normalized natural variable, its lower bound, and the resulting polynomial or linear inequality. No new tactic or broad automatic simplifier is implemented here.

For this project, useful candidates would have an existing local scalar induction and a direct downstream consumer. Replacing a readable one-line library theorem by new infrastructure would have no benefit. Earlier generic advice to keep large powers symbolic remains part of the assumed baseline.

7.3 When is the explicit quotient representation worth adopting?

The optional decoder may become more valuable if later code computes with quotient words, tests exact tile indices, or needs an extraction-friendly representative. For purely propositional consumers already using `quotientWord_tile`, the existing choice-based definition is compact and the 13 additional lines may not pay for themselves. A useful next check is one actual computational client, with both the equality proof and any reduction-sensitive behavior examined.

References

[1] ProveIt, final exclusion cutoff `09d65a93c9df8684c1a226341d6a5120623d1cdb`. Packet w; four incoming archives and index. The closing audit records each archive identity and the comparisons with its proposals.

[2] ProveIt, placed proposal index and adoption record, proof-source revision `29936089835c0c55cca81edc86b4fb0c70ce34d5`. Pinned proposal collection.

[3] ProveIt, intermediate a–v exclusion revision `efa64ca941cdae37bb6f7fc5093ae8bb69296ca3`. Packet v; incoming index.

[4] ProveIt, `SelectiveCoefficientBounds.lean`, source revision above. Pinned source.

[5] ProveIt, `SplitCoefficientBounds.lean`. Pinned source.

[6] ProveIt, `NativePellPower.lean`. Pinned source.

[7] ProveIt, `NativeHalfBinomial.lean`. Proof-source version; closing revision, target declaration unchanged.

[8] ProveIt, `WangTiles942Permutation.lean`. Decoder and table injectivity.

[9] ProveIt, `WangTiles942HelicalQuotient.lean`. Baseline quotient construction.